

Aspect Coordination Rules as Substrate-Resident Operational Specification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher) **ORCID:** 0009-0004-8065-3235 **Date:** May 8, 2026 **License:** Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in *The Instinct/Reasoning Separation Outside the Model* (Li, April 2026), the second paper in the CKS theory series following *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone derivation, the operational specification of how aspects coordinate constituent cells through substrate-resident coordination rules — the second of five notes decomposing Paper 2’s commitment that an aspect is a coordination arrangement of cells.

Abstract

Paper 2 names the aspect as a “coordination arrangement of cells serving a particular purpose.” A companion derivation note formalizes the purpose-defined character of that arrangement. This note formalizes the second component: the operational specification of how the aspect actually coordinates its constituent cells. That specification takes the form of substrate-resident coordination rules — authored by humans under the authority architecture inherited from Paper 1, recorded with provenance, governable, inspectable, and modifiable as substrate content. The note states what those rules cover (membership, invocation, output-integration, cross-cell conflict-handling, purpose-alignment, lifecycle), distinguishes substrate-resident coordination from the implicit coordination logic that conventional AI orchestrators encode in code and configuration, identifies the inherited Paper 1 commitments the specification operates under, and provides an operational test for whether a given aspect’s coordination is substrate-resident in the CKS sense.

1. Why aspect coordination rules need to be formalized as standalone operational specification

Paper 2 introduces three architectural levels — cell, aspect, Self — and locates the aspect at the middle scope: a coordination arrangement of cells serving a particular purpose. A companion derivation note formalizes the purpose-defined character of the arrangement: what the aspect is *for*. What that companion does not formalize, and what this note formalizes, is the operational specification of how the aspect actually coordinates its constituent cells once the purpose is set.

A purpose statement is necessary but not sufficient. Stating that an aspect is for, say, “competitive sports mode” or “calm study mode” does not tell us which cells participate, when the aspect invokes them, how their outputs combine, what happens when their outputs disagree, or when membership changes. The operational specification of those coordination behaviors is a separable architectural object. Naming it as such allows it to be governed independently of the purpose, audited at its own scope, and evolved through Paper 2’s evolution mechanisms — directed selection through DNA evolution and refinement through action-feedback.

Conventional AI architectures often leave this layer implicit. The “orchestrator” coordinates components through code paths or configuration files: a conditional that decides when to call a sub-agent, a routing

table that maps request types to models, a workflow definition that encodes step ordering as graph topology. Each is, in some sense, coordination logic; none is substrate-resident authoritative content under human governance in the CKS sense. The CKS commitment — that coordination logic is itself substrate content under the same authority architecture as everything else — needs naming as a standalone derivation to keep the slide into implicit-coordination patterns visible. This is the second of five notes decomposing Paper 2’s “aspect as coordination arrangement” commitment into independently governable architectural objects.

2. The specification, precisely stated

In the CKS pattern, aspect coordination operates through **substrate-resident coordination rules** authored under the authority architecture inherited from Paper 1. These rules cover six operational concerns, each separable, each governable as substrate content.

Membership rules specify which cells participate as members of the aspect, by direct assignment or by pattern-based inclusion. Membership rules are themselves substrate content under the same authority architecture; a companion derivation formalizes membership as a separate operational object.

Invocation rules specify when and how the aspect calls on its constituent cells: which cell to invoke for which aspect-level question, what inputs to pass, what outputs to expect. Invocation typically follows Paper 1’s Pattern A (consultation) — the aspect consults cells as authoritative sources for content within their scope.

Output-integration rules specify how constituent-cell outputs are combined to form aspect-level outputs: aggregation, selection, transformation, filtering, ranking. Integration typically follows Paper 1’s Pattern B (derived view) — the aspect derives views over cell substrate content rather than assuming command authority over the cells.

Cross-cell conflict-handling rules specify how the aspect treats disagreement among its constituent cells. The category is structurally constrained: Paper 1’s conflict-as-first-class commitment requires that conflicts be registered as substrate state and surfaced for governance, not silently merged into a single output. The rules specify the registration mechanism, the surfacing path, and the governance escalation; they do not specify a default auto-resolution.

Purpose-alignment rules specify how aspect operations are checked against the purpose specification a companion derivation note formalizes. Misalignment surfaces as a governance event rather than as a silent failure; the rules specify what counts as alignment in the deployment’s terms.

Lifecycle rules specify how aspect operations trigger membership changes, and how the aspect itself transitions through lifecycle events, connecting the aspect’s coordination behavior to the broader birth/mating/death machinery Paper 2 develops at every level.

All six categories are substrate-resident authoritative content under Paper 1’s authority architecture, with authorship and modification history recorded as substrate metadata. A change to any rule is itself a governance event, recorded with Paper 1’s six-field provenance vocabulary (who, when, under what authority, what changed, what rationale, what conflicts).

3. What makes the specification architecturally distinctive

The architectural distinctiveness of substrate-resident coordination rules is best seen in contrast with conventional AI orchestration patterns.

Conventional AI orchestration: implicit coordination. A typical multi-component AI system encodes coordination logic somewhere in its implementation — a function that routes requests, a configuration file that defines a workflow graph, a table that maps intents to handlers, a prompt template that decides when to delegate to sub-agents. Each is real coordination logic, but none is content under the same authority architecture as the substrate it coordinates. Inspecting it requires reading source code or configuration files, presupposing engineering skill; modifying it requires changing the implementation, a different kind of action than editing substrate content; auditing it produces verifiable answers only for someone who can read the code.

CKS: explicit substrate-resident coordination. The CKS commitment is that coordination logic is itself substrate content, and therefore subject to the same authority architecture as the rest of the substrate: inspection uses the same inspect right that grounds substrate inspection generally; modification uses the same modify right; override uses the same override right. The coordination layer is not a separate engineering surface with its own access patterns and its own authority structure; it is substrate content with the structural status of substrate content.

The consequences are concrete. Coordination changes are governance events with provenance; the question “who changed how the aspect coordinates its cells, when, under what authority, with what rationale” has the same kind of substrate-grounded answer as the corresponding question about any other substrate write. Coordination behavior is explainable through inspection of the applicable rules rather than through code review; non-specialist humans can audit it in the same environments where they audit substrate content; coordination rule conflicts can be registered as first-class substrate state when they arise. This is the property that distinguishes substrate-resident coordination from the implicit-coordination patterns that surround it.

4. Inherited Paper 1 commitments

Aspect coordination rules operate under a set of architectural commitments inherited directly from Paper 1, with no modification at the aspect scope.

Composition and patterns. Cell-aspect composition satisfies Paper 1’s composition constraints, and Paper 1’s three composition patterns — consultation (A), derived view (B), separate concerns (C) — are the patterns aspect coordination uses. Invocation typically follows A; integration typically follows B; aspects whose constituent cells operate independently with the aspect integrating outputs typically use C.

Authority, content, provenance. Coordination rules are authored by humans under Paper 1’s authority architecture; LLMs may draft them, but humans hold the authority over what takes effect. The rules are substrate-resident authoritative content — not derived views, not cached projections, not advisory annotations. Coordination events — every invocation, every integration, every conflict registration — are recorded with Paper 1’s six-field provenance vocabulary, showing which rules applied for which aspect operation.

Determinism. Paper 1’s determinism contract holds at the aspect scope: given the same inputs and the same coordination rules, an aspect operation produces the same coordination behavior. Variability in cell-level outputs (for example, LLM-generated content within a cell) is preserved as cell-level variability and surfaces at the aspect’s output-integration step under the integration rules.

Conflict as first-class, and human-governed. Paper 1’s conflict-preservation commitment binds the cross-cell conflict-handling rules: aspects do not auto-resolve cross-cell conflicts; they register them as substrate state and surface them for governance. Paper 1’s human-governed commitment applies to coordination rules as it applies to substrate content generally — humans retain the rights to inspect, modify, and override coordination rules at all times.

These commitments are inherited, not extended. The aspect-scope specification adds operational structure within them; it does not relax or weaken them.

5. Operational implications

Several implications follow directly from the specification.

Deployments configure coordination per aspect purpose, and patterns may differ across aspects in the same Self. Different purposes call for different patterns: an aspect organized around fast pattern-matching may use Pattern A with simple integration; an aspect organized around deliberate reasoning may use Pattern B derived views with conflict-preserving integration. The configuration is a deployment decision, recorded as substrate content, modifiable under the authority architecture; the architecture does not require uniform coordination across aspects.

Coordination rules evolve through both directed selection and action-feedback. Paper 2’s DNA evolution mechanism — human-governed orchestration substrate updates — applies to coordination rules at the aspect scope; operators refine coordination through intentional design improvements, recorded as governance events with provenance. Paper 2’s action-feedback evolution mechanism applies in parallel: patterns observed across aspect operations surface as candidate refinements, proposed under governance rather than auto-applied.

Coordination is testable, and cross-partner deployments preserve authority boundaries. The reproducibility commitment Paper 1 grounds at the substrate layer applies at the aspect scope: given recorded inputs and the coordination rules in effect at the time, coordination behavior can be reconstructed by replay through the substrate. When an aspect coordinates cells under different authority scopes — a multi-organizational deployment — Paper 1’s authority-distribution architecture grounds the coordination, and coordination rules respect the authority boundaries the deployment defines.

Vertical evolution modifies coordination through aspect restructuring. When Paper 2’s vertical evolution restructures composition — cells reassigned across aspects, aspects split or merged or dissolved — coordination rules update accordingly; the structural change is itself substrate content under the authority architecture. A cell may also participate in multiple aspects simultaneously, each with its own coordination rules; the cell’s content is the same, but the coordination context differs per aspect, and a companion derivation formalizes multiple-aspect participation as a separable architectural object.

6. Limits

Naming the specification precisely also requires naming what it does not commit to.

Coordination rules do not command cells, and they do not bypass the substrate-cell boundary. The aspect’s relationship to its constituent cells is content-domain reasoning, not command authority: the aspect coordinates cells through inspection, consultation, derivation, and integration. Paper 1’s substrate-cell boundary is preserved at the aspect scope; cells maintain their substrate boundaries under their own cell-scope governance.

Coordination rules do not auto-resolve conflicts, and they do not eliminate cell-level governance. The conflict-as-first-class commitment binds the cross-cell conflict-handling rules: the aspect does not silently merge disagreeing cell outputs; it registers the conflict and surfaces it for governance. Cells, in turn, remain governed at the cell scope; aspect coordination is an additional governance layer over cell composition, not a displacement of the governance that grounds the cells themselves.

Coordination rules do not prescribe specific patterns, and they are not exhaustive of aspect operations. The architecture commits to substrate-resident coordination as a property; it does not prescribe which combination of Pattern A, Pattern B, and Pattern C any particular aspect must use. The aspect also has a purpose specification (formalized separately), membership records, lifecycle states, and provenance metadata; coordination rules are one component of the aspect’s substrate footprint, not the whole of it.

Coordination rules at multiple aspects do not conflict by default. When the same cell participates in multiple aspects with different coordination contexts, the differences are not conflicts; they are scope-distinct rules. Cross-aspect rule conflicts, if they arise, are governed by Paper 1’s rule conflict resolution architecture.

7. Operational test

A system’s aspect coordination is substrate-resident in the CKS sense if and only if all of the following hold at all times during the aspect’s existence:

1. The rules governing aspect membership, invocation, output-integration, cross-cell conflict-handling, purpose-alignment, and lifecycle are held as substrate content readable in inspectable form, not only as code paths or configuration files.
2. The rules are authored under the same authority architecture as substrate content generally; LLM-drafted rules are admissible only when subject to human authority before they take effect.
3. Changes to any rule are recorded as governance events with the six-field provenance vocabulary Paper 1 commits substrate writes to.
4. Aspect coordination operations record provenance showing which rules applied to which operation.
5. Cross-cell conflicts are registered as substrate state and surfaced for governance, not silently merged into a single output.
6. No LLM operation, vendor policy, or runtime middleware can in principle prevent humans from inspecting, modifying, or overriding the coordination rules.
7. Coordination operations are reproducible by replay given the recorded rules and inputs.

A system that fails any of (1)–(7) may have an “orchestrator” that coordinates cells in some sense, but its coordination is not substrate-resident in the CKS sense, and downstream work that relies on that property should be scoped accordingly.

8. Conclusion

Naming aspect coordination rules as substrate-resident operational specification gives the aspect’s coordination behavior the same architectural status as the rest of the substrate it operates over. Coordination logic is content; the content is authored, governed, recorded, inspectable, and modifiable; coordination changes are governance events; coordination behavior is explainable through inspection of applicable rules. This is what distinguishes the CKS commitment from the implicit-coordination patterns that conventional AI orchestrators rely on, and it is what makes coordination evolvable through Paper 2’s evolution mechanisms at the aspect’s own scope.

This note is the second of five notes decomposing Paper 2’s “aspect as coordination arrangement” commitment. The first names what the aspect is for; this one names how the aspect operates. Subsequent notes formalize the aspect-cell content-domain relationship, the multiple-aspect cell participation pattern, and inherited-commitment verification at the aspect scope, after which Phase B2 turns to the Self level.

Subsequent work that implements, extends, or argues against the CKS aspect coordination commitment should use “aspect coordination rules” in the sense formalized here. Subsequent work that uses the term

differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Predecessor paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Aspect Coordination Rules as Substrate-Resident Operational Specification in the Coordination Knowledge Substrate Pattern*. May 8, 2026. ORCID: 0009-0004-8065-3235.